RWFS User Manual

Relativistic WaveFunction Solver -
bound-state solutions of the radial Dirac equation

Based on: Mansour, Gamal, Fritzsche & El Sherbini,
"RWFS: An open-source Python package for bound-state solutions of the radial Dirac equation in arbitrary central-field potentials"

Repository: https://github.com/atoscopy/rwfs

Contents

1 What RWFS Does **3**

2 Installation **3**

3 Quick Start **4**

4 Theoretical Background **4**
4.1 The radial Dirac equation 4
4.2 Power-series (Salvat) method 5

5 Central-Field Potentials **5**
5.1 Finite nuclear size (options 8 and 9) 6

6 Radial Grids **8**

7 Program Architecture **9**

8 Running RWFS - Interactive Mode **10**

9 Running RWFS - Batch Mode **11**

10 Input File Reference **12**

11 Output File Reference **13**

12 Orbital Notation Reference **13**

13 Exporting Orbitals to GRASP **14**

14 Radial Expectation Values **14**

15 Choosing a Potential and Grid **14**

16 Validation Summary **15**

17 Troubleshooting / FAQ **17**

1 What RWFS Does

RWFS solves the **radial Dirac equation** for a bound electron moving in any spherically symmetric (central) potential, and returns:

- the orbital energy $\varepsilon_{n\kappa}$,
- the normalized large and small radial components $P_{n\kappa}(r)$ and $Q_{n\kappa}(r)$,
- radial expectation values such as $\langle r \rangle$, $\langle r^2 \rangle$, $\langle 1/r \rangle$.

It can generate a **complete set of bound orbitals in one run** (e.g. every occupied shell of a neutral atom), rather than one orbital per execution as in most legacy Fortran solvers. Orbitals can be written out in RWFS's own ASCII format or converted to the binary format read by **GRASP**, so RWFS is a convenient way to generate starting orbitals for MCDHF / RCI calculations.

RWFS is a good fit when you need:

- a quick, disposable set of relativistic orbitals for a model potential (Coulomb, Thomas–Fermi, Yukawa, screened-hydrogenic, finite-size nuclear models, or your own tabulated potential),
- a lightweight cross-check against a B-spline or finite-difference DHF code,
- starting orbitals to hand to GRASP for a full MCDHF/RCI calculation.

It does **not** solve the many-electron SCF problem itself (no exchange potential, no self-consistency loop) - for that, feed RWFS's output, or a converged SCF potential, in as the central field, or use it purely as an independent-particle / model-potential tool.

2 Installation

```
git clone https://github.com/atoscopy/rwfs.git
cd rwfs
```

Requirements

- Python $\geq$ 3.0 (no external numerical dependencies beyond the standard library are required for the core solver; the package is written in pure Python)
- A Fortran compiler (e.g. `gfortran`) **only** if you plan to use the GRASP-export utility `readrwf_grasp.f` in `tools/`

Directory layout

```
rwfs/
|-- src/
|   |-- rwfs.py          # main driver
|   |-- constants.py     # physical constants, unit conversions
|   |-- grid.py          # geometric / exponential radial grids
```

```
|   |-- potential.py     # central-field potential models
|   |-- spline.py        # natural cubic-spline interpolation
|   |-- mathematic.py    # linear-algebra helpers (tridiagonal solver)
|   |-- energy_loop.py   # shooting-and-matching Dirac solver
|-- tools/
|   |-- readrwf_grasp.f  # converts RWFS output to GRASP binary format
|-- examples/
    |-- example_2/        # batch run: neutral argon
    |-- example_4/        # interactive run: neutral aluminium
    |-- ...
```

No build step is needed to run the solver itself - `rwfs.py` is executed directly with the system Python interpreter.

3 Quick Start

Interactive, single-session calculation:

```
python src/rwfs.py
```

then answer the prompts (see Section 8 for the full transcript). For a reproducible, non-interactive run:

```
python src/rwfs.py < input-ar > output-ar
```

where `input-ar` supplies every answer RWFS would otherwise ask for interactively (see Section 9).

4 Theoretical Background

4.1 The radial Dirac equation

For an electron in a spherically symmetric potential $V(r)$, the large and small radial components $P_{n\kappa}(r)$, $Q_{n\kappa}(r)$ satisfy the coupled first-order system (atomic units, $m_e = e = \hbar = 1$):

$$\begin{pmatrix} c\left(\dfrac{d}{dr} + \dfrac{\kappa}{r}\right) & -\varepsilon_{n\kappa} + V(r) - 2c^2 \\ \varepsilon_{n\kappa} - V(r) & c\left(\dfrac{d}{dr} - \dfrac{\kappa}{r}\right) \end{pmatrix} \begin{pmatrix} P_{n\kappa}(r) \\ Q_{n\kappa}(r) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{1}$$

with c the speed of light and κ the relativistic angular quantum number:

$$\kappa = \begin{cases} -(l+1) & j = l + \frac{1}{2} \\ l & j = l - \frac{1}{2} \end{cases} \tag{2}$$

Normalization is fixed by

$$\int_0^\infty \left[P_{n\kappa}(r)^2 + Q_{n\kappa}(r)^2\right] dr = 1 \tag{3}$$

4.2 Power-series (Salvat) method

RWFS integrates these equations with **Salvat's power-series method** [1, 2] rather than finite differences or B-splines [3]:

1. The radial domain is split into intervals (r_i, r_{i+1}); within each, a local variable $x = (r - r_i)/h$ maps the interval to $[0, 1]$.
2. P and Q are expanded as power series in x, and substituting into the transformed Dirac equations gives a **recurrence relation** for the series coefficients (they are generated term-by-term, not solved as a linear system).
3. The potential is represented locally by a **natural cubic spline**, so the recurrence coefficients stay simple (up to cubic terms) even for a numerically tabulated $V(r)$.
4. Because the recurrence is singular at the origin, the **first interval** uses a modified expansion $P \sim x^s$, $Q \sim x^{s+t}$, with s, t fixed analytically by the near-origin behavior of $V(r)$ and κ - this correctly reproduces the $r^{\gamma-1}$ (point nucleus) or regular polynomial (finite nucleus) behavior at small r (see Section 5.1).
5. **Shooting and matching**: the regular solution is integrated *outward* from the origin; a solution satisfying the correct exponential decay is integrated *inward* from the asymptotic region; the two are matched at an intermediate point, and the orbital energy is corrected iteratively (first-order energy correction) until the matching residual converges.

This approach avoids the two classic headaches of finite-basis relativistic calculations - spurious states and explicit basis-set completeness - at the cost of being restricted to bound states only (no continuum).

5 Central-Field Potentials

RWFS ships with the following nine built-in models. All are selected by number at the interactive prompt or by keyword in a batch input file.

#	Name	Formula	Typical use
1	Thomas–Fermi (TF)	statistical screening function $\phi(m)$	quick, parameter-free screened potential for any atom/ion
2	Coulomb	$V(r) = -Z/r$	hydrogen-like ions; inner-shell benchmark
3	Yukawa	$V(r) = -(Z/r)\, e^{-\alpha r}$	simple exponential screening

#	Name	Formula	Typical use
4	Multi-Yukawa	$V(r) = -(Z/r)\sum_i c_i e^{-\alpha_i r}$	best analytic fit to DHF core orbitals (Salvat et al. parameters) [4]
5	Screened Coulomb	$V(r) = -\frac{1}{r}\left[1 + (Z-1)e^{-\alpha r}\right]$	nuclear charge at short range, ion charge at long range
6	Screened hydrogenic	$V(r) = -Z_{\text{eff}}/r = -(Z-\sigma)/r$	crude, fast effective-charge model
7	User-defined (tabulated)	2-column $(r, V(r))$ ASCII file	any externally generated potential - SCF, DFT, HF, MCDHF
8	Uniformly charged sphere	$\rho(r) = 3Z/(4\pi R^3)$ for $r \le R$, $R = \sqrt{5/3}\,R_{\text{rms}}$ (Section 5.1)	finite nuclear size, high-Z hydrogen-like ions
9	Fermi charge distribution	$\rho(r) = \rho_0/\{1 + \exp[(r-c)/a]\}$, $a = t/(4\ln 3)$, c fixed by R_{rms} (Section 5.1)	realistic finite nuclear size [5]

Practical guidance

- **Coulomb** systematically *overbinds* everything except true one-electron ions - screening is completely absent.
- **Thomas–Fermi** gives a good general-purpose screened potential and tracks GRASP's own TF starting potential closely (this is the best choice if you want RWFS energies to agree with a GRASP starting calculation).
- **Multi-Yukawa** (option 4) with the Salvat et al. parameters gives the best agreement with self-consistent DHF for *inner-shell* orbitals, but can noticeably underbind or even nearly unbind diffuse valence orbitals - treat outer-shell energies from this potential with caution.
- **User-defined** is the right choice whenever you already have a converged SCF/DFT potential (e.g. from a B-spline DHF code) and want RWFS to regenerate the corresponding orbital set or extend it to additional n, κ.

5.1 Finite nuclear size (options 8 and 9)

Options 1–6 treat the nucleus as a point charge. For high-Z ions the finite extent of the nuclear charge matters: the s and $p_{1/2}$ levels penetrate the nucleus, and their binding energies change by tens to hundreds of eV for $Z \sim 80$–105 (see Section 16). Options 8 and 9 generate the potential of an extended, spherically symmetric nuclear charge distribution. Only the bare nuclear potential is produced (no electron screening), exactly like the Coulomb option 2, so these options are intended for hydrogen-like ions and as the nuclear part of a model potential.

General relations. For a spherical charge density $\rho(r)$ normalized to the nuclear charge,

$$4\pi \int_0^\infty \rho(r)\, r^2\, dr = Z, \tag{4}$$

the potential energy of an electron is

$$V(r) = -\left[\frac{Q(r)}{r} + S(r)\right], \qquad Q(r) = 4\pi\int_0^r \rho(r')\, r'^2 dr', \quad S(r) = 4\pi\int_r^\infty \rho(r')\, r'\, dr', \tag{5}$$

where $Q(r)$ is the charge inside radius r and $S(r)$ is the potential of the shells outside it. RWFS works with $rV(r)$,

$$rV(r) = -\big[\, Q(r) + r\, S(r)\,\big], \tag{6}$$

which satisfies $rV(0) = 0$ (the potential stays finite at the origin) and $rV(r) \to -Z$ outside the nucleus. The model parameters are fixed by the root-mean-square charge radius,

$$\langle r^2\rangle = \frac{4\pi}{Z}\int_0^\infty \rho(r)\, r^4\, dr = R_{\rm rms}^2. \tag{7}$$

Because $rV(0) = 0$, the first integration interval uses the $u_0 = 0$ start-up expansions (cases ii and iii of the origin series) instead of the point-Coulomb case.

Option 8: uniformly charged sphere.

$$\rho(r) = \frac{3Z}{4\pi R^3}\ (r \le R), \qquad \rho(r) = 0\ (r > R), \qquad \langle r^2\rangle = \tfrac{3}{5}R^2 \ \Rightarrow\ R = \sqrt{\tfrac{5}{3}}\, R_{\rm rms}. \tag{8}$$

Inserting in Eq. (6):

$$rV(r) = \begin{cases} -Z\left[\dfrac{3r}{2R} - \dfrac{r^3}{2R^3}\right], & r \le R, \\ -Z, & r > R. \end{cases} \tag{9}$$

rV and its first derivative are continuous at $r = R$.

Option 9: Fermi (two-parameter Woods–Saxon) distribution.

$$\rho(r) = \frac{\rho_0}{1 + \exp\left[(r-c)/a\right]} \equiv \rho_0 f(r), \qquad a = \frac{t}{4\ln 3}, \tag{10}$$

where c is the half-density radius and a the diffuseness, fixed by the 10%–90% surface thickness t (default $t = 2.3$ fm, i.e. $a = 0.5234$ fm). With $M_n = \int_0^\infty r^n f(r)\, dr$ and the inversion formula of the polylogarithm, the moments needed are, in closed form,

$$M_2 = \frac{c^3}{3} + \frac{\pi^2 a^2 c}{3} + 2a^3\, e_3, \tag{11}$$

$$M_4 = \frac{c^5}{5} + \frac{2\pi^2 a^2 c^3}{3} + \frac{7\pi^4 a^4 c}{15} + 24a^5\, e_5, \tag{12}$$

$$e_s = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k^s}\, e^{-kc/a}, \tag{13}$$

where the terms $e_s \sim e^{-c/a} \approx 10^{-5}$ are retained, so the expressions are exact. The normalization and the rms radius follow as

$$\rho_0 = \frac{Z}{4\pi M_2}, \qquad \langle r^2\rangle = \frac{M_4}{M_2} = \tfrac{3}{5}\, c^2 + \tfrac{7}{5}\,\pi^2 a^2 + O\Big(e^{-c/a}\Big). \tag{14}$$

For a given $R_{\rm rms}$, RWFS solves $M_4(c)/M_2(c) = R_{\rm rms}^2$ for c by bisection (the ratio is monotonic in c); the leading-order estimate is $c \simeq \sqrt{\frac{5}{3}R_{\rm rms}^2 - \frac{7}{3}\pi^2 a^2}$. This requires $R_{\rm rms} > \sqrt{7/5}\,\pi a \simeq 1.95$ fm for $t = 2.3$ fm; for lighter nuclei RWFS stops with an error message and the uniform sphere (option 8) should be used. Inserting Eq. (10) into Eq. (6) gives

$$rV(r) = -\frac{Z}{M_2}\Big[\,I_2(r) + r\,J_1(r)\Big], \qquad I_2(r) = \int_0^r x^2 f(x)\,dx, \quad J_1(r) = \int_r^\infty x\,f(x)\,dx. \tag{15}$$

I_2 is accumulated forward and J_1 backward over the radial grid using 8-point Gauss–Legendre quadrature on sub-panels no wider than $a/2$; beyond $r_{\rm cut} = c + 60a$, where $f < 10^{-26}$, $rV = -Z$ is set exactly.

Units and input. Lengths are entered in femtometres and converted with 1 fm $= 1/a_0$ a.u., $a_0 =$ 52917.7210903 fm. Both this constant and the default skin thickness are defined in `constants.py` (`a0_fm`, `fermi_skin_t_fm`). After choosing option 8 or 9, RWFS asks for the rms radius in one of two ways:

- enter the nuclear mass number A, and RWFS estimates

$$R_{\rm rms} = 0.836\,A^{1/3} + 0.570 \text{ fm} \tag{16}$$

  (global fit of Johnson and Soff, valid for $A > 9$; use a measured radius whenever one is available), or

- press Enter and type $R_{\rm rms}$ in fm directly.

Option 9 then asks for the skin thickness t in fm (Enter keeps 2.3 fm). The run header in `summary.dat` records $R_{\rm rms}$ and, for option 8, the sphere radius R (in fm and a.u.), and for option 9 the parameters t, c and a (fm).

6 Radial Grids

Two nonuniform radial grids are available:

Geometric grid (default)

$$r_i = dr \cdot r_{i-1} \qquad (dr > 1,\ \text{constant ratio}) \tag{17}$$

Dense near the nucleus, progressively coarser at large r.

Exponential grid (GRASP-compatible)

$$r_i = r_0\left[e^{h(i-1)} - 1\right], \qquad r_0 = T/Z \tag{18}$$

Identical in form to GRASP's own radial grid, so wavefunctions generated with this option can be compared point-by-point with GRASP output - this is the grid to use for the GRASP cross-checks described in Section 16.

Choosing the number of points, `np`

A convergence study was performed for the $7s$ orbital of neutral uranium using the Thomas–Fermi potential. The calculated energy and radial moments were found to be stable to all reported digits for `np` ≥ 1800, with no observable changes as the number of points was increased further up to `np` $= 3000$. In contrast, calculations with `np` $= 1700$–1750 exhibited a small residual drift. Based on these results, `np` $=$ **3000** is adopted as the default value in RWFS, providing a comfortable margin for numerical convergence at negligible computational cost. Even at this grid density, a single-orbital calculation typically takes less than ~ 0.6 s.

7 Program Architecture

```
rwfs.py  (main driver)
   |-- grid.py         -> radial grid construction
   |-- potential.py    -> central potential V(r) on the grid
   |-- constants.py    -> physical constants / unit conversions
   |-- energy_loop.py  -> shooting-and-matching Dirac solver
         |-- mathematic.py  -> tridiagonal solver (linear algebra kernel)
         |-- spline.py      -> natural cubic-spline interpolation
```

Module	Responsibility
`rwfs.py`	Reads input (interactive or batch), builds the grid and potential, invokes the solver for each requested orbital, prints/writes the summary and wavefunction files.
`constants.py`	Physical constants, atomic-unit conversion factors (a.u. $\leftrightarrow$ eV, etc.), numerical tolerances used throughout, and the nuclear-model constants (a_0 in fm, default Fermi skin thickness t).
`grid.py`	Geometric and exponential grid generation and manipulation.
`potential.py`	All nine potential models listed in Section 5, including the tabulated-file reader and its cubic-spline construction, and the finite-nuclear-size models (uniformly charged sphere, Fermi distribution) with the mass-number estimate of R_{rms}.
`spline.py`	Natural cubic-spline interpolation, used both for the potential representation inside each integration interval and for evaluating radial expectation values after convergence.
`mathematic.py`	Numerical linear-algebra routines, in particular the tridiagonal solver used internally by the Dirac solver.
`energy_loop.py`	The core physics: outward/inward power-series integration, wavefunction matching, and the iterative energy-correction loop until the convergence criterion is met.

8 Running RWFS - Interactive Mode

Interactive mode is for exploration and testing input parameters. Start it with no arguments:

```
python src/rwfs.py
```

Example transcript - occupied orbitals of neutral aluminium ($Z = 13$) using the Thomas–Fermi potential:

```
$ python src/rwfs.py

Enter the atomic number (Z) of the atom:
13

Enter the number of electrons in the atom/ion:
13

Enter the orbital you want either as
(1) spectroscopic notation or
(2) as (n,kappa) notation :
1

Enter ALL the orbitals you want:
1s 2s 2p- 2p 3s 3p- 3p

Do you want to change the default parameters (y/n)?
n

Choose the central-field potential:
1 - Thomas-Fermi
2 - Coulomb
3 - Yukawa
4 - Multi-Yukawa
5 - Screened
6 - Screened-hydrogenic
7 - Read potential from file
8 - Uniformly charged sphere (finite nucleus)
9 - Fermi charge distribution (finite nucleus)
Enter the potential index:
1
```

After the run converges, RWFS prints a summary table directly to the terminal:

```
atomic number (Z) = 13
number of electrons (Ne) = 13
potential used = Thomas-Fermi potential

Orbital   n   kappa      E (a.u.)         E (eV)
--------------------
1s        1    -1     -54.927961615    -1494.667
2s        2    -1      -4.905396535     -133.483
2p-       2     1      -3.595387315      -97.836
```

```
2p        2    -2      -3.578467390      -97.375
3s        3    -1      -0.562389516      -15.303
3p-       3     1      -0.327442398       -8.910
3p        3    -2      -0.326281219       -8.879

Orbitals NOT converged:
(none -- all requested orbitals converged)
```

and writes the radial functions to `radial.rwf` in the working directory.

Orbital input: you can type orbitals either in spectroscopic notation (`1s 2s 2p- 2p ...`, using RWFS's compact minus-sign convention - see Section 12) or as explicit (n, κ) pairs, whichever you find easier to script.

9 Running RWFS - Batch Mode

For reproducible or scripted calculations, redirect a prepared input file into RWFS instead of typing answers by hand:

```
python src/rwfs.py < input-ar > output-ar
```

- `input-ar` contains, in order, every value the interactive prompts above would ask for: Z, N_e, the orbital list, whether to change default numerical parameters, and the potential choice (plus any potential-specific parameters, e.g. screening constants or a filename for a tabulated potential).
- `output-ar` captures the full run log (identical in content to what would print to the terminal in interactive mode).

This makes it trivial to drive RWFS from a shell loop or a Python wrapper for systematic studies (isoelectronic sequences, grid-convergence scans, etc.) - this is exactly how the paper generated its $Z = 1$–105 hydrogenic validation table and its neutral-atom convergence study.

Files produced by a batch run

(example: `examples/example_2`, neutral argon):

File	Content
`output-ar`	complete execution log
`summary.dat`	orbital quantum numbers, energies (a.u. and eV), radial expectation values, potential used, convergence status
`radial.rwf`	large/small radial components, ASCII
`grasp_Ar`	orbitals in GRASP-compatible ASCII format
`potential_Ar`	the central potential itself, tabulated on the radial grid

10 Input File Reference

A batch input file supplies, one answer per line, exactly what the interactive session asks for:

```
Z                      # atomic number
Ne                     # number of bound electrons
1                      # orbital-input mode: 1 = spectroscopic, 2 = (n,kappa)
1s 2s 2p- 2p 3s ...    # orbital list (mode 1) -- or "n kappa" pairs (mode 2)
n                      # change default numerical parameters? y/n
                       #   if y: follow with grid type, np, dr or h, etc.
<potential index>      # 1-9, see Section 5
<potential parameters> # only if the chosen potential needs extra input:
                       #   - Yukawa: screening parameter alpha
                       #   - Multi-Yukawa: number of terms + (c_i, alpha_i) pairs
                       #   - Screened / screened-hydrogenic: alpha or sigma
                       #   - option 7: path to the two-column (r, V) file
                       #   - options 8, 9: mass number A, or an empty line
                       #     followed by R_rms in fm; option 9 then asks
                       #     for the skin thickness t in fm (empty = 2.3)
```

Example: finite-nucleus Au^{78+} (batch input). Hydrogen-like gold ($Z = 79$, one electron), the $1s_{1/2}$, $2s_{1/2}$ and $2p_{1/2}$ orbitals, Fermi nucleus with the rms radius estimated from the mass number $A = 197$ and the default skin thickness $t = 2.3$ fm:

```
79
1
1
1s 2s 2p-
n
9
197
2.3
```

The lines after `9` are the mass number and the skin thickness t (in fm). To give $R_{\rm rms}$ directly, leave the mass-number line *empty* and type the radius in fm on the next line; to keep the default t, leave the last line empty. (Empty lines are significant in a batch input file.) The summary written to the terminal and to `summary.dat` then lists the nuclear parameters actually used:

```
potential used = Fermi charge distribution (finite nucleus)
                 (R_rms = 5.43438961722746 fm, t = 2.3 fm)
potential constants:
  Z          = 79
  R_rms (fm) = 5.43438961722746
  t (fm)     = 2.3
  c (fm)     = 6.550765238587178
  a (fm)     = 0.5233875553104315

Orbital   n  kappa        E (a.u.)        E (eV)
-------------------
1s        1    -1   -3.4327827801E+03  -9.341081E+04
2s        2    -1   -8.7892389901E+02  -2.391675E+04
2p-       2     1   -8.7920572606E+02  -2.392442E+04
```

User-defined potential file format (option 7): a plain two-column ASCII file,

```
r_1   V(r_1)
r_2   V(r_2)
...
```

RWFS builds a natural cubic spline through these points and evaluates it during integration - so the file doesn't need to be on RWFS's own radial grid, but should cover the full range you intend to integrate over and be dense enough to resolve any near-origin curvature.

11 Output File Reference

File	Written by	Contents
`radial.rwf`	every run	ASCII large/small radial components $P(r)$, $Q(r)$ for each converged orbital
`summary.dat`	batch mode	orbital n, κ, energies (a.u. & eV), $\langle r\rangle$, $\langle r^2\rangle$, potential used, per-orbital convergence flag
`grasp_<name>`	on request / batch	GRASP-compatible ASCII orbital file (input to `readrwf_grasp.f`)
`potential_<name>`	on request / batch	the central potential $V(r)$ tabulated on the radial grid actually used

Radial expectation values are all evaluated from the converged, normalized wavefunction via

$$\langle f(r)\rangle = \int_0^\infty \left[P(r)^2 + Q(r)^2\right] f(r)\, dr \tag{19}$$

so any operator you can express as $f(r)$ - not just $\langle r\rangle$, $\langle r^2\rangle$, $\langle 1/r\rangle$ - can in principle be evaluated from the stored radial functions.

12 Orbital Notation Reference

RWFS uses a compact ASCII notation for relativistic subshells: a trailing minus sign marks the $j = l - 1/2$ member of a spin–orbit doublet; no suffix means $j = l + 1/2$.

Spectroscopic	RWFS	l	j	κ
$s_{1/2}$	`s`	0	1/2	-1
$p_{1/2}$	`p-`	1	1/2	$+1$
$p_{3/2}$	`p`	1	3/2	-2
$d_{3/2}$	`d-`	2	3/2	$+2$
$d_{5/2}$	`d`	2	5/2	-3
$f_{5/2}$	`f-`	3	5/2	$+3$
$f_{7/2}$	`f`	3	7/2	-4

General rule: $\kappa = -(l+1)$ for $j = l + 1/2$, and $\kappa = +l$ for $j = l - 1/2$.

13 Exporting Orbitals to GRASP

The `tools/readrwf_grasp.f` utility (a modified version of the standard GRASP `readrwf` tool [6]) converts the ASCII `rwf.all` file produced by RWFS into the binary orbital format GRASP expects.

1. Compile it with any Fortran compiler, e.g.:

```
gfortran -o readrwf_grasp tools/readrwf_grasp.f
```

2. Run it against your RWFS output (consult the README shipped alongside `readrwf_grasp.f` for the exact invocation and for adjusting internal array dimensions if you need an unusually large radial grid).
3. The resulting binary file can be used directly as a set of starting orbitals for a GRASP MCDHF or RCI calculation [6, 7]

This lets you use RWFS purely as a fast, flexible **starting-orbital generator** for otherwise-standard GRASP workflows - e.g. generating a Multi-Yukawa-screened initial guess that converges faster than GRASP's built-in hydrogenic starting potential.

14 Radial Expectation Values

Beyond the orbital energy, RWFS automatically evaluates, for every converged orbital:

- $\langle r \rangle$ - mean radius
- $\langle r^2 \rangle$ - mean-square radius
- $\langle 1/r \rangle$ - inverse radius (relevant to hyperfine and isotope-shift work)

using cubic-spline interpolation of the converged wavefunction (Section 4.2). These are written to `summary.dat` in batch mode and reported at the end of an interactive run if requested.

15 Choosing a Potential and Grid

Quick decision guide:

- **Testing the solver itself / need an exact reference** → Coulomb potential, any Z; compare directly against the analytic Dirac energies (Section 16).
- **Rough starting orbitals for a neutral atom or ion, no other code available** → Thomas–Fermi.
- **Best inner-shell accuracy without a full SCF calculation** → Multi-Yukawa (Salvat parameters) [4]; be cautious with outer valence shells.
- **You already have a self-consistent potential** (from DBSR_HF, GRASP, a DFT code, ...) → user-defined tabulated potential; this is the most accurate option available and effectively turns RWFS into a single-electron Dirac solver for *your* SCF field.
- **High-Z ion where finite nuclear size matters (energies of s and $p_{1/2}$ levels)** → Fermi distribution (option 9, the standard model) or uniform sphere (option 8); compare with the Coulomb result to see the size of the effect (Section 5.1).
- **Need output on the same grid as GRASP** → exponential grid ($r_i = r_0[e^{h(i-1)} - 1]$, $r_0 = T/Z$).
- **Otherwise** → default geometric grid with `np` = 3000 is a safe, fast default per the convergence study in Section 16.

16 Validation Summary

Hydrogen-like ions (Coulomb potential), $Z = 1$–105. Comparing $1s_{1/2}$, $2p_{1/2}$, $3d_{3/2}$, $4f_{5/2}$ against the exact analytic Dirac energies:

- absolute errors stay below 5×10^{-7} a.u. across the entire range,
- relative errors are typically $\sim 10^{-11}$ and never exceed 1.2×10^{-10},
- accuracy is essentially independent of Z and of orbital angular momentum / node count.

Finite nuclear size, hydrogen-like ions from H to Db^{104+}. Each ion was calculated with the point nucleus (option 2), the uniformly charged sphere (option 8) and the Fermi distribution (option 9, $t = 2.3$ fm). The nuclear radii were taken from the mass-number fit $R_{\rm rms} = 0.836\,A^{1/3} + 0.570$ fm with $A = 1$, 4, 12, 40, 139, 197 and 268 for H, He, C, Ar, La, Au and Db; they are illustrative and should be replaced by the isotope and radius of interest. The point-nucleus results reproduce the exact Dirac energies to relative errors between 10^{-16} and 5×10^{-11}, as in the Coulomb benchmark above. The finite-size shifts $E_{\rm finite} - E_{\rm point}$ and the difference $\delta_{\rm FS} = E_{\rm Fermi} - E_{\rm sphere}$ are listed in Table 1; they are the same data as in the paper. Shifts are quoted only to the digits resolved by the 11-digit energies printed by RWFS, and ~ 0 marks values below three times that resolution. The $3d$ and $4f$ levels are unaffected at this resolution.

For H and He the mass-number fit lies outside its range of validity ($A > 9$); use measured radii for such light nuclei.

Neutral argon, comparing RWFS's analytic potentials against self-consistent DHF results from JAC, FAC, and a B-spline DHF code: core-orbital ($1s$, $2s$, $2p$) energies from the Multi-Yukawa

Table 1: Finite-nuclear-size shifts in eV (all runs converged). Numbers in the form $a \times 10^b$ are small shifts. [a] The sphere and Fermi runs for H and He used different radii (the Fermi model needs $R_{\rm rms} > 1.95$ fm for $t = 2.3$ fm), so $\delta_{\rm FS}$ is not meaningful there.

Ion	$R_{\rm rms}$ (fm)	Orbital	$\Delta E_{\rm sph}$	$\Delta E_{\rm Fermi}$	$\delta_{\rm FS}$
H	1.406 / 1.950[a]	$1s_{1/2}$	1.3×10^{-8}	2.5×10^{-8}	–
		$2s_{1/2}$	2×10^{-9}	3.0×10^{-9}	–
		$2p_{1/2}$	~ 0	~ 0	–
		$2p_{3/2}$	~ 0	~ 0	–
He^{+}	1.897 / 1.950[a]	$1s_{1/2}$	3.76×10^{-7}	3.97×10^{-7}	–
		$2s_{1/2}$	4.65×10^{-8}	4.93×10^{-8}	–
		$2p_{1/2}$	~ 0	~ 0	–
		$2p_{3/2}$	~ 0	~ 0	–
C^{5+}	2.484	$1s_{1/2}$	5.263×10^{-5}	5.263×10^{-5}	~ 0
		$2s_{1/2}$	6.591×10^{-6}	6.591×10^{-6}	~ 0
		$2p_{1/2}$	~ 0	~ 0	~ 0
		$2p_{3/2}$	~ 0	~ 0	~ 0
Ar^{17+}	3.429	$1s_{1/2}$	9.049×10^{-3}	9.046×10^{-3}	-3.3×10^{-6}
		$2s_{1/2}$	1.149×10^{-3}	1.148×10^{-3}	-4.1×10^{-7}
		$2p_{1/2}$	3.76×10^{-6}	3.76×10^{-6}	~ 0
		$2p_{3/2}$	~ 0	~ 0	~ 0
La^{56+}	4.901	$1s_{1/2}$	4.563	4.557	-6.321×10^{-3}
		$2s_{1/2}$	0.6662	0.6653	-9.260×10^{-4}
		$2p_{1/2}$	0.02407	0.02404	-3.18×10^{-5}
		$2p_{3/2}$	~ 0	~ 0	~ 0
Au^{78+}	5.434	$1s_{1/2}$	49.18	49.09	-0.09171
		$2s_{1/2}$	8.332	8.317	-0.01562
		$2p_{1/2}$	0.6489	0.6477	-1.171×10^{-3}
		$2p_{3/2}$	5.4×10^{-6}	6.0×10^{-6}	~ 0
Db^{104+}	5.960	$1s_{1/2}$	778.2	776.6	-1.597
		$2s_{1/2}$	169.4	169.0	-0.3489
		$2p_{1/2}$	29.71	29.65	-0.06026
		$2p_{3/2}$	9.3×10^{-5}	9.8×10^{-5}	~ 0

potential agree best with the self-consistent calculations; the unscreened Coulomb potential substantially overbinds everything; all analytic potentials show larger, model-dependent discrepancies for the valence $3s/3p$ shells, with Multi-Yukawa in particular nearly failing to bind $3p$.

Neutral lanthanum vs. GRASP, same Thomas–Fermi potential in both codes: agreement is at the level of independent-implementation numerical noise across all 20 occupied orbitals from $1s$ to $6s$ - this is the strongest available cross-check of the solver itself (as opposed to the choice of potential model). Radial wavefunctions (not just energies) for representative orbitals of C, Ar, and La are visually indistinguishable from GRASP over the full radial domain, including the near-nuclear region and the asymptotic tail.

Grid convergence (7s of U, $Z = 92$, Thomas–Fermi): energy and radial moments converge and stop changing for `np` ≥ 1800; default `np` $= 3000$ gives comfortable margin.

Performance: single-orbital $1s$ calculations across H ($Z = 1$) through Og ($Z = 118$) all complete in well under one second on a single desktop CPU core, with only weak dependence on Z.

17 Troubleshooting / FAQ

An orbital doesn't converge ("Orbitals NOT converged" in the summary).
Try (a) increasing `np` (grid density), (b) switching from the geometric to the exponential grid or vice versa, (c) checking that the requested (n, κ) is physically sensible for the given potential (a very diffuse valence orbital in a strongly screened potential can be only weakly bound or even unbound - see the Multi-Yukawa $3p$ example in Section 16).

Energies look far too negative / too shallow compared to another code.
Check which central potential you selected - Coulomb vs. Thomas–Fermi vs. Multi-Yukawa can differ by an order of magnitude for outer shells of a many-electron atom, purely because of how much screening each model assumes (Section 5). This is a physical/model difference, not a solver bug.

I want RWFS energies to match GRASP as closely as possible.
Use the Thomas–Fermi potential and the exponential (GRASP-style) radial grid - this combination is what the lanthanum benchmark (Section 16) validates against GRASP directly.

Continuum / scattering states.
Not supported - RWFS is bound-state only. Planned future work (per the paper) includes continuum-state support, additional central-field models, and improved interfaces to other relativistic atomic-structure codes.

Are finite-nuclear-size potentials supported?
Yes. Options 8 (uniformly charged sphere) and 9 (Fermi distribution) generate the bare nuclear potential for a given rms radius (Section 5.1). For a many-electron model potential, generate the nuclear-plus-screening potential externally with a finite nucleus and pass it in as a tabulated file (option 7); the file must be dense enough to resolve the nuclear region ($r \lesssim 10^{-4}$ a.u.).

Option 9 stops with "R_rms is too small for a Fermi distribution".
For $t = 2.3$ fm the Fermi distribution requires $R_{\rm rms} > 1.95$ fm. Use the uniform sphere (option 8) for light nuclei, or reduce the skin thickness t.

I have a tabulated SCF potential from my own code - can I feed it in?
Yes - option 7 (user-defined potential), a two-column ASCII $(r, V(r))$ file; RWFS builds a natural cubic spline internally (Section 10).

References

[1] F. Salvat and R. Mayol, "Accurate numerical solution of the Schrödinger and Dirac wave equations for central fields," *Computer Physics Communications*, vol. 62, no. 1, pp. 65–79, 1991.

[2] F. Salvat and J. M. Fernandez-Varea, "Radial: A fortran subroutine package for the solution of the radial Schrödinger and Dirac wave equations," *Computer Physics Communications*, vol. 240, pp. 165–177, 2019.

[3] O. Zatsarinny and C. F. Fischer, "Dbsr_hf: A b-spline dirac–hartree–fock program," *Computer Physics Communications*, vol. 202, pp. 287–303, 2016.

[4] F. Salvat, J. D. Martínez, R. Mayol, and J. Parellada, "Analytical dirac-hartree-fock-slater screening function for atoms ($z = 1$–92)," *Physical Review A*, vol. 36, no. 2, pp. 467–474, 1987.

[5] F. A. Parpia and A. K. Mohanty, "Relativistic basis-set calculations for atoms with fermi nuclei," *Physical Review A*, vol. 46, no. 7, pp. 3735–3745, 1992.

[6] P. Jönsson, G. Gaigalas, J. Bieroń, C. F. Fischer, and I. P. Grant, "New version: Grasp2k relativistic atomic structure package," *Computer Physics Communications*, vol. 184, no. 9, pp. 2197–2203, 2013.

[7] C. F. Fischer, G. Gaigalas, P. Jönsson, and J. Bieroń, "GRASP2018–a fortran 95 version of the general relativistic atomic structure package," *Computer Physics Communications*, vol. 237, pp. 184–187, 2019.

Repository: https://github.com/atoscopy/rwfs